Higher Order Functions

```
Functions is a first class citizens, 4 rules satisfied
1. Assigned to a variable
2. Passed as an argument
3. Returned from another function
4. Can be modified
```

Functions as Parameters

```
def function(parameter):

def higher_order_function(function, parameter):
    return function(parameter)
```

Function as a Return Value

```
def function1():
def function2():
def higher_order_function():
    if condition:
        return function1
    else:
        return function2
```

Python Closures

```
a nested function is able to access the outer scopte of the enclosing
function, known as closure
```

```
def add_ten():
    ten = 10
    def add(num):
        return num + ten
```

Python Decorators

```
- design pattern to add new functionality without modifying its structure,
called before the original function
- you need inner wrapper to have the same functionality of receiving
paramters
```

```
def decorator(function):
    def wrapper():
        func = function()
        make_uppercase = func.upper()
        return make_uppercase

    return wrapper

# You can use multiple decorator
@decorator1
@decorator2
def greeting():
    return "Welcome to python"
print(greeting())
# or

g = decorator(greeting)
print(g())
```

Built-in Higher Order Function

| map() function

```
apply function to every item in an iterable
```

```
numbers = [1, 2, 3]
squared = map(lambda x:x**2, numbers)
print(list(squared))
```

| filter() function

```
filter item by returning boolean to filtering criteria
```

```
def is_even(num):
    if num % 2 == 0:
        return True
    else:
        return Flase

even_numbers = filter(is_even, numbers)
```

| reduce() function

```
defined in functools module, need to import
return a single value, take only two arguments
```

```
def add_two_numbers(x, y):
    return int(x) + int(y)

total = reduce(add_two_numbers, numbers)
# Workflow: (1+2)+3
```